МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №7

"Робота з базами даних у Python"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Навчитися працювати з SQLite, SQL, DB-API, параметризованими запитами, SQLAlchemy ORM, Repository, транзакціями, ACID, Alembic та integration tests у попередньому Python-проєкті.

Теоретичні відомості:

Persistence layer зберігає стан між запусками. Repository приховує деталі запитів від сервісу, ORM пов'язує об'єкти Python із таблицями, а транзакції об'єднують зміни в одну операцію. Міграції послідовно змінюють схему. Integration tests перевіряють реальну взаємодію ізольованої SQLite-бази, моделей, Repository і сервісу.

Виконання роботи

1. Варіант 1: постановка задачі та предметна область

Продовжив систему обліку студентів із попередніх робіт. Зберігаю ім'я, прізвище, групу та середній бал у SQLite. Відокремив роботу з БД від бізнес-логіки.

• Продовжити проєкт Lab_01 без зміни предметної області. Для студента зберігати: ім'я, прізвище, групу, середній бал.

• Створити пов'язані таблиці groups і students (зв'язок one-to-many). Реалізувати StudentRepository та GroupRepository.

• Зберегти функціональність Lab_01: додавання і виведення студентів, пошук за прізвищем, пошук за групою, визначення найкращого студента та середнього бала групи. Додати зміну середнього бала, переведення між групами та видалення.

• Через DB-API реалізувати параметризований пошук студентів за групою. Транзакційна операція: масове переведення студентів до іншої групи з commit/rollback.

• Міграція Alembic: додати поле email до students. Integration tests: CRUD Student, робота Group, filter_by_group, агрегація середнього бала та rollback транзакції.

2. Структура проєкту

```
pyproject.toml
alembic.ini
src/student_manager/main.py
src/student_manager/lab7_demo.py
src/student_manager/persistence/database.py
src/student_manager/persistence/models.py
src/student_manager/persistence/repositories.py
src/student_manager/persistence/service.py
src/student_manager/persistence/dbapi.py
src/student_manager/persistence/migrations/env.py
src/student_manager/persistence/migrations/versions/0001_initial.py
src/student_manager/persistence/migrations/versions/0002_student_email.py
tests/test_persistence.py
tests/test_persistence_atomic_validation.py
tests/test_migrations.py
tests/test_dbapi.py
tests/test_lab7_demo.py
tests/test_lab7_cli.py
tests/test_migration_target.py
```

3. Схема БД, таблиці, поля та ключі

src/student_manager/persistence/models.py - GroupRow

```python
class GroupRow(Base):
    # orm сутність таблиці groups
    __tablename__ = "groups"
    __table_args__ = (
        CheckConstraint("length(trim(name)) > 0", name="check_group_name_nonempty"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(
        String(100), unique=True, nullable=False, index=True
    )

    students: Mapped[list[StudentRow]] = relationship(
        back_populates="group", passive_deletes=True
    )

    def to_dto(self) -> GroupDTO:
        # створення незмінного dto об'єкта
        return GroupDTO(id=self.id, name=self.name)
```

src/student_manager/persistence/models.py - StudentRow

```python
class StudentRow(Base):
    # orm сутність таблиці students
    __tablename__ = "students"
    __table_args__ = (
        CheckConstraint(
            "average_grade >= 0.0 AND average_grade <= 100.0",
            name="check_student_average_grade_range",
        ),
        CheckConstraint(
            "length(trim(first_name)) > 0",
            name="check_student_first_name_nonempty",
        ),
        CheckConstraint(
            "length(trim(last_name)) > 0",
            name="check_student_last_name_nonempty",
        ),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    first_name: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
    last_name: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
    average_grade: Mapped[float] = mapped_column(Float, nullable=False)
    group_id: Mapped[int] = mapped_column(
        ForeignKey("groups.id", ondelete="RESTRICT"), nullable=False, index=True
    )
    email: Mapped[str | None] = mapped_column(String(255), nullable=True, default=None)

    group: Mapped[GroupRow] = relationship(back_populates="students")

    def to_student(self, group_name: str | None = None) -> Student:
        # конвертація orm рядка в доменний dataclass
        grp = (
            group_name
            if group_name is not None
            else (self.group.name if self.group else "")
        )
        return Student(
            first_name=self.first_name,
            last_name=self.last_name,
            group=grp,
            average_grade=self.average_grade,
            student_id=self.id,
        )

    def to_dto(self, group_name: str | None = None) -> StudentDTO:
        # конвертація orm рядка в деталізований dto
        grp = (
            group_name
            if group_name is not None
            else (self.group.name if self.group else "")
        )
        return StudentDTO(
            id=self.id,
            first_name=self.first_name,
            last_name=self.last_name,
            group_id=self.group_id,
            group_name=grp,
            average_grade=self.average_grade,
            email=self.email,
        )
```

.local_workflow/lab7/capture_report_evidence.py - inspect_sqlite_database

```python
def inspect_sqlite_database(db_path: Path) -> dict[str, Any]:
    conn = sqlite3.connect(str(db_path))
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    # check foreign keys pragma
    cursor.execute("PRAGMA foreign_keys;")
    fk_pragma = cursor.fetchone()[0]

    # query sqlite_master
    cursor.execute(
        "SELECT type, name, tbl_name, sql FROM sqlite_master "
        "WHERE type IN ('table', 'index', 'trigger') "
        "ORDER BY type, name;"
    )
    sqlite_master = [dict(row) for row in cursor.fetchall()]

    # table names
    cursor.execute(
        "SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' ORDER BY name;"
    )
    tables = [row["name"] for row in cursor.fetchall()]

    table_info: dict[str, list[dict[str, Any]]] = {}
    foreign_keys: dict[str, list[dict[str, Any]]] = {}
    indexes: dict[str, list[dict[str, Any]]] = {}
    ddl: dict[str, str] = {}

    for tbl in tables:
        # DDL
        cursor.execute("SELECT sql FROM sqlite_master WHERE type='table' AND name=?;", (tbl,))
        row = cursor.fetchone()
        ddl[tbl] = row["sql"] if row else ""
```

```python
        # columns
        cursor.execute(f"PRAGMA table_info({tbl});")
        table_info[tbl] = [dict(r) for r in cursor.fetchall()]

        # FKs
        cursor.execute(f"PRAGMA foreign_key_list({tbl});")
        foreign_keys[tbl] = [dict(r) for r in cursor.fetchall()]

        # indexes
        cursor.execute(f"PRAGMA index_list({tbl});")
        indexes[tbl] = [dict(r) for r in cursor.fetchall()]

    # live data
    cursor.execute("SELECT id, name FROM groups ORDER BY id;")
    groups_data = [dict(r) for r in cursor.fetchall()]

    cursor.execute(
        "SELECT id, first_name, last_name, group_id, average_grade, email FROM students ORDER BY id;"
    )
    students_data = [dict(r) for r in cursor.fetchall()]

    conn.close()

    return {
        "sqlite_version": sqlite3.sqlite_version,
        "foreign_keys_pragma": fk_pragma,
        "tables": tables,
        "sqlite_master": sqlite_master,
        "ddl": ddl,
        "table_info": table_info,
        "foreign_keys": foreign_keys,
        "indexes": indexes,
        "live_data": {
            "groups": groups_data,
```

```
            "students": students_data,
        },
    }
```

Зв'язок groups.id (1) -> students.group_id (N). Первинні ключі groups.id і students.id унікально визначають записи. Зовнішній ключ students.group_id посилається на groups.id; ON DELETE RESTRICT забороняє видалити заповнену групу. Службова таблиця alembic_version зберігає поточну ревізію.

Таблиця	Поле	Тип	Ключ	NULL
groups	id	INTEGER	PK	NOT NULL

Таблиця	Поле	Тип	Ключ	NULL
groups	name	VARCHAR(100)		NOT NULL
students	id	INTEGER	PK	NOT NULL
students	first_name	VARCHAR(100)		NOT NULL
students	last_name	VARCHAR(100)		NOT NULL
students	average_grade	FLOAT		NOT NULL
students	group_id	INTEGER	FK -> groups.id	NOT NULL
students	email	VARCHAR(255)		NULL дозволено
alembic_version	version_num	VARCHAR(32)	PK	NOT NULL

Таблиця	Поле	Призначення
groups	id	Унікальний числовий ідентифікатор групи; на нього посилається students.group_id.
groups	name	Унікальна непорожня назва навчальної групи; використовую для відображення та DB-API пошуку.
students	id	Унікальний числовий ідентифікатор студента для CRUD і переведення між групами.
students	first_name	Ім'я студента; порожнє значення або лише пробіли не допускаються.
students	last_name	Прізвище студента; використовую для пошуку, порожнє значення не допускається.
students	average_grade	Середній бал за шкалою від 0 до 100 включно; використовую для рейтингу та середнього бала групи.
students	group_id	Ідентифікатор наявної групи з groups.id; визначає належність студента до групи.
students	email	Необов'язкова електронна адреса студента; NULL означає, що адресу не вказано. Поле додала міграція 0002.
alembic_version	version_num	Ідентифікатор застосованої ревізії Alembic для відстеження поточного стану схеми.

4. SQL-команди створення таблиць

src/student_manager/persistence/migrations/versions/0001_initial.py - upgrade

```python
def upgrade() -> None:
    # створення початкових таблиць groups та students без колонки email
    op.create_table(
        "groups",
        sa.Column("id", sa.Integer(), autoincrement=True, nullable=False),
        sa.Column("name", sa.String(length=100), nullable=False),
        sa.CheckConstraint("length(trim(name)) > 0", name="check_group_name_nonempty"),
        sa.PrimaryKeyConstraint("id"),
        sa.UniqueConstraint("name"),
    )
    op.create_index(op.f("ix_groups_name"), "groups", ["name"], unique=True)

    op.create_table(
        "students",
        sa.Column("id", sa.Integer(), autoincrement=True, nullable=False),
        sa.Column("first_name", sa.String(length=100), nullable=False),
        sa.Column("last_name", sa.String(length=100), nullable=False),
        sa.Column("average_grade", sa.Float(), nullable=False),
        sa.Column("group_id", sa.Integer(), nullable=False),
        sa.CheckConstraint(
            "average_grade >= 0.0 AND average_grade <= 100.0",
            name="check_student_average_grade_range",
        ),
        sa.CheckConstraint(
            "length(trim(first_name)) > 0",
            name="check_student_first_name_nonempty",
        ),
        sa.CheckConstraint(
            "length(trim(last_name)) > 0",
            name="check_student_last_name_nonempty",
        ),
        sa.ForeignKeyConstraint(["group_id"], ["groups.id"], ondelete="RESTRICT"),
        sa.PrimaryKeyConstraint("id"),
    )
```

```python
    op.create_index(
        op.f("ix_students_first_name"), "students", ["first_name"], unique=False
    )
    op.create_index(
        op.f("ix_students_group_id"), "students", ["group_id"], unique=False
    )
    op.create_index(
        op.f("ix_students_last_name"), "students", ["last_name"], unique=False
    )
```

src/student_manager/persistence/migrations/versions/0002_student_email.py - upgrade

```python
def upgrade() -> None:
    # додавання опціонального стовпця email до таблиці students
    with op.batch_alter_table("students", schema=None) as batch_op:
        batch_op.add_column(sa.Column("email", sa.String(length=255), nullable=True))
```

Нижче навів справжній DDL зі збереженого sqlite_master після міграцій, а не повторний запуск CREATE TABLE.

```sql
CREATE TABLE groups (
    id INTEGER NOT NULL,
    name VARCHAR(100) NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT check_group_name_nonempty CHECK (length(trim(name)) > 0),
    UNIQUE (name)
)
```

```sql
CREATE TABLE students (
    id INTEGER NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    average_grade FLOAT NOT NULL,
    group_id INTEGER NOT NULL, email VARCHAR(255),
    PRIMARY KEY (id),
    CONSTRAINT check_student_average_grade_range CHECK (average_grade >= 0.0 AND average_grade <= 100.0),
    CONSTRAINT check_student_first_name_nonempty CHECK (length(trim(first_name)) > 0),
    CONSTRAINT check_student_last_name_nonempty CHECK (length(trim(last_name)) > 0),
    FOREIGN KEY(group_id) REFERENCES groups (id) ON DELETE RESTRICT
)
```

```
CREATE TABLE alembic_version (
    version_num VARCHAR(32) NOT NULL,
    CONSTRAINT alembic_version_pkc PRIMARY KEY (version_num)
)
```

5. Налаштування engine та Session

src/student_manager/persistence/database.py

```
from __future__ import annotations

from collections.abc import Generator
from contextlib import contextmanager
from typing import Any

from sqlalchemy import Engine, create_engine, event
from sqlalchemy.orm import Session, sessionmaker


def create_database_engine(url: str, echo: bool = False, **kwargs: Any) -> Engine:
    # створення рушія sqlalchemy з налаштуванням sqlite foreign keys
    engine = create_engine(url, echo=echo, **kwargs)

    if engine.dialect.name == "sqlite":

        @event.listens_for(engine, "connect")
        def _enable_sqlite_foreign_keys(
            dbapi_connection: Any, connection_record: Any
        ) -> None:
            cursor = dbapi_connection.cursor()
            cursor.execute("PRAGMA foreign_keys=ON")
            cursor.close()
            if hasattr(dbapi_connection, "create_function"):
                dbapi_connection.create_function(
                    "lower",
                    1,
                    lambda s: s.casefold() if s is not None else None,
                )

    return engine


def create_session_factory(engine: Engine) -> sessionmaker[Session]:
```

```
    # створення фабрики сесій без застарівання об'єктів після коміту
    return sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)


@contextmanager
def session_scope(
    session_factory: sessionmaker[Session],
) -> Generator[Session, None, None]:
    # контекстний менеджер життєвого циклу сесії з транзакцією
    session = session_factory()
    try:
        yield session
        if session.in_transaction():
            session.commit()
    except Exception:
        if session.in_transaction():
            session.rollback()
        raise
    finally:
        session.close()
```

Рушій вмикає PRAGMA foreign_keys для кожного власного SQLite-з'єднання. Session factory створює незалежні сесії; session_scope виконує commit після успіху, rollback після винятку та закриває сесію.

У збереженій перевірці схеми окреме з'єднання sqlite3 мало PRAGMA foreign_keys=0; це не з'єднання engine. Увімкнення FK у власному engine перевіряє integration test нижче.

6. Repository Pattern і CRUD

src/student_manager/persistence/repositories.py - GroupRepository.create

```python
def create(self, name: str) -> GroupDTO:
    # створення нової групи
    if not isinstance(name, str):
        raise TypeError("Group name must be a string")
    trimmed = name.strip()
    if not trimmed:
        raise ValueError("Group name cannot be empty")

    row = GroupRow(name=trimmed)
    self.session.add(row)
    self.session.flush()
    return row.to_dto()
```

src/student_manager/persistence/repositories.py - GroupRepository.get

```python
def get(self, group_id: int) -> GroupDTO | None:
    # пошук групи за числовим id
    _validate_positive_int(group_id, "group_id")
    row = self.session.get(GroupRow, group_id)
    return row.to_dto() if row is not None else None
```

src/student_manager/persistence/repositories.py - GroupRepository.list

```python
def list(self) -> builtins.list[GroupDTO]:
    # повертає список усіх груп з детермінованим сортуванням
    stmt = select(GroupRow).order_by(GroupRow.name.asc(), GroupRow.id.asc())
    rows = self.session.execute(stmt).scalars().all()
    return [row.to_dto() for row in rows]
```

src/student_manager/persistence/repositories.py - GroupRepository.delete

```python
def delete(self, group_id: int) -> bool:
    # видалення групи за id
    _validate_positive_int(group_id, "group_id")
    row = self.session.get(GroupRow, group_id)
    if row is None:
        return False
    self.session.delete(row)
    self.session.flush()
    return True
```

src/student_manager/persistence/repositories.py - StudentRepository.create

```python
def create(
    self,
    student: Student,
    group_id: int | None = None,
    email: str | None = None,
) -> Student:
    # створення нового студента в прив'язці до групи
    if not isinstance(student, Student):
        raise TypeError("student must be an instance of Student")

    if group_id is None:
        group_row = self.session.execute(
            select(GroupRow).where(
                func.lower(GroupRow.name) == student.group.strip().casefold()
            )
        ).scalar_one_or_none()
        if group_row is None:
            raise ValueError(f"Group '{student.group}' not found")
    else:
        _validate_positive_int(group_id, "group_id")
        group_row = self.session.get(GroupRow, group_id)
        if group_row is None:
            raise ValueError(f"Group with id {group_id} not found")

    email_val: str | None = None
    if email is not None:
        if not isinstance(email, str):
            raise TypeError("email must be a string")
        email_val = email.strip() or None

    row = StudentRow(
        first_name=student.first_name,
        last_name=student.last_name,
        average_grade=float(student.average_grade),
```

```python
        group_id=group_row.id,
        email=email_val,
    )
    if student.student_id is not None:
        _validate_positive_int(student.student_id, "student_id")
        row.id = student.student_id

    self.session.add(row)
    self.session.flush()
    return row.to_student(group_row.name)
```

src/student_manager/persistence/repositories.py - StudentRepository.get

```python
def get(self, student_id: int) -> Student | None:
    # отримання доменного об'єкта студента за id
    _validate_positive_int(student_id, "student_id")
    stmt = (
        select(StudentRow)
        .options(joinedload(StudentRow.group))
        .where(StudentRow.id == student_id)
    )
    row = self.session.execute(stmt).unique().scalar_one_or_none()
    return row.to_student() if row is not None else None
```

src/student_manager/persistence/repositories.py - StudentRepository.list

```python
def list(self, offset: int = 0, limit: int = 100) -> builtins.list[Student]:
    # пагінований список студентів за зростанням id
    if type(offset) is bool or not isinstance(offset, int) or offset < 0:
        raise ValueError("offset must be a non-negative integer")
    if type(limit) is bool or not isinstance(limit, int) or limit <= 0:
        raise ValueError("limit must be a positive integer")

    stmt = (
        select(StudentRow)
        .options(joinedload(StudentRow.group))
        .order_by(StudentRow.id.asc())
        .offset(offset)
        .limit(limit)
    )
    rows = self.session.execute(stmt).unique().scalars().all()
    return [row.to_student() for row in rows]
```

src/student_manager/persistence/repositories.py - StudentRepository.update

```python
def update(
    self,
    student_id: int,
    *,
    first_name: str | None = None,
    last_name: str | None = None,
    group_id: int | None = None,
    average_grade: float | None = None,
    email: str | None = None,
) -> Student | None:
    # оновлення окремих полів студента
    _validate_positive_int(student_id, "student_id")
    row = self.session.get(StudentRow, student_id)
    if row is None:
        return None

    validated_first = row.first_name
    validated_last = row.last_name
    validated_group = row.group_id
    validated_grade = row.average_grade
    validated_email = row.email
    if first_name is not None:
        if not isinstance(first_name, str):
            raise TypeError("first_name must be a string")
        validated_first = first_name.strip()
        if not validated_first:
            raise ValueError("first_name cannot be empty")
    if last_name is not None:
        if not isinstance(last_name, str):
            raise TypeError("last_name must be a string")
        validated_last = last_name.strip()
        if not validated_last:
            raise ValueError("last_name cannot be empty")
    if group_id is not None:
```

```
            _validate_positive_int(group_id, "group_id")
            if self.session.get(GroupRow, group_id) is None:
                raise ValueError(f"Group with id {group_id} not found")
            validated_group = group_id
        if average_grade is not None:
            if type(average_grade) is bool or not isinstance(
                average_grade, (int, float)
            ):
                raise TypeError("average_grade must be a number")
            validated_grade = float(average_grade)
            if not 0.0 <= validated_grade <= 100.0:
                raise ValueError("Average grade must be between 0 and 100")
        if email is not None:
            if not isinstance(email, str):
                raise TypeError("email must be a string")
            validated_email = email.strip() or None

        row.first_name = validated_first
        row.last_name = validated_last
        row.group_id = validated_group
        row.average_grade = validated_grade
        row.email = validated_email

        self.session.flush()
        target_grp = self.session.get(GroupRow, row.group_id)
        grp_name = target_grp.name if target_grp is not None else ""
        return row.to_student(grp_name)
```

src/student_manager/persistence/repositories.py - StudentRepository.delete

```
def delete(self, student_id: int) -> bool:
    # видалення студента за id
    _validate_positive_int(student_id, "student_id")
    row = self.session.get(StudentRow, student_id)
    if row is None:
        return False
    self.session.delete(row)
    self.session.flush()
    return True
```

Repository виконує запити та flush, а сервіс задає межі транзакції. Додавання, читання, зміну й видалення перевірив у демонстрації та integration tests.

7. Пошук, фільтрація та SQL-агрегація

src/student_manager/persistence/repositories.py - StudentRepository.search_by_last_name

```
def search_by_last_name(self, surname: str) -> builtins.list[Student]:
    # пошук за прізвищем без урахування регістру та зайвих пробілів
    if not isinstance(surname, str):
        raise TypeError("surname must be a string")
    trimmed = surname.strip()
    if not trimmed:
        raise ValueError("surname cannot be empty")

    stmt = (
        select(StudentRow)
        .options(joinedload(StudentRow.group))
        .where(func.lower(func.trim(StudentRow.last_name)) == trimmed.casefold())
        .order_by(StudentRow.id.asc())
    )
    rows = self.session.execute(stmt).unique().scalars().all()
    return [row.to_student() for row in rows]
```

src/student_manager/persistence/repositories.py - StudentRepository.filter_by_group

```python
def filter_by_group(self, group_id: int) -> builtins.list[Student]:
    # фільтрація студентів конкретної групи
    _validate_positive_int(group_id, "group_id")
    stmt = (
        select(StudentRow)
        .options(joinedload(StudentRow.group))
        .where(StudentRow.group_id == group_id)
        .order_by(StudentRow.id.asc())
    )
    rows = self.session.execute(stmt).unique().scalars().all()
    return [row.to_student() for row in rows]
```

src/student_manager/persistence/repositories.py - StudentRepository.best

```python
def best(self, group_id: int | None = None) -> Student | None:
    # найкращий студент з детермінованим tie-breaker
    if group_id is not None:
        _validate_positive_int(group_id, "group_id")
    stmt = select(StudentRow).options(joinedload(StudentRow.group))
    if group_id is not None:
        stmt = stmt.where(StudentRow.group_id == group_id)
    stmt = stmt.order_by(
        StudentRow.average_grade.desc(), StudentRow.id.asc()
    ).limit(1)
    row = self.session.execute(stmt).unique().scalar_one_or_none()
    return row.to_student() if row is not None else None
```

src/student_manager/persistence/repositories.py - StudentRepository.average

```python
def average(self, group_id: int | None = None) -> float | None:
    # обчислення середнього балу через sql avg
    if group_id is not None:
        _validate_positive_int(group_id, "group_id")
    stmt = select(func.avg(StudentRow.average_grade))
    if group_id is not None:
        stmt = stmt.where(StudentRow.group_id == group_id)
    result = self.session.execute(stmt).scalar_one_or_none()
    return float(result) if result is not None else None
```

src/student_manager/persistence/repositories.py - StudentRepository.count

```python
def count(self, group_id: int | None = None) -> int:
    # кількість студентів через sql count
    if group_id is not None:
        _validate_positive_int(group_id, "group_id")
    stmt = select(func.count(StudentRow.id))
    if group_id is not None:
        stmt = stmt.where(StudentRow.group_id == group_id)
    return int(self.session.execute(stmt).scalar_one_or_none() or 0)
```

8. DB-API та parameterized query

src/student_manager/persistence/dbapi.py

```python
from __future__ import annotations

import sqlite3
from pathlib import Path

from student_manager.models import Student


def _casefold_str(value: str | None) -> str | None:
    # юнікодний регістронезалежний переклад для sqlite
    return value.casefold() if value is not None else None


def get_connection(database: str | Path) -> sqlite3.Connection:
    # створення sqlite з'єднання з активними fk та функцією lower
    conn = sqlite3.connect(str(database))
    conn.execute("PRAGMA foreign_keys = ON")
    conn.create_function("lower", 1, _casefold_str)
    return conn


def find_students_by_group(
    database: str | Path | sqlite3.Connection,
    group_name: str,
) -> list[Student]:
    # параметризований пошук студентів за назвою групи через db-api
    if not isinstance(group_name, str):
        raise TypeError("Group name must be a string")

    trimmed = group_name.strip()
    if not trimmed:
        return []

    owns_connection = False
```

```python
    if isinstance(database, (str, Path)):
        conn = get_connection(database)
        owns_connection = True
    else:
        conn = database
        try:
            conn.create_function("lower", 1, _casefold_str)
        except sqlite3.OperationalError:
            # функція вже могла бути зареєстрована раніше
            pass

    try:
        cursor = conn.cursor()
        try:
            query = """
                SELECT s.id, s.first_name, s.last_name, g.name, s.average_grade
                FROM students s
                JOIN groups g ON s.group_id = g.id
                WHERE lower(g.name) = lower(?)
                ORDER BY s.id ASC
            """
            cursor.execute(query, (trimmed,))
            rows = cursor.fetchall()
            return [
                Student(
                    student_id=row[0],
                    first_name=row[1],
                    last_name=row[2],
                    group=row[3],
                    average_grade=float(row[4]),
                )
                for row in rows
            ]
        finally:
```

```
            cursor.close()
    finally:
        if owns_connection:
            conn.close()
```

.local_workflow/lab7/capture_report_evidence.py - execute_dbapi_queries

```python
def execute_dbapi_queries(db_path: Path) -> dict[str, Any]:
    conn = sqlite3.connect(str(db_path))
    conn.execute("PRAGMA foreign_keys = ON;")
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    query_sql = (
        "SELECT s.id, s.first_name, s.last_name, g.name AS group_name, s.average_grade, s.email "
        "FROM students s "
        "JOIN groups g ON s.group_id = g.id "
        "WHERE lower(g.name) = lower(?) "
        "ORDER BY s.id;"
    )

    # normal query
    target_group = "ФеП-33"
    cursor.execute(query_sql, (target_group,))
    normal_rows = [dict(r) for r in cursor.fetchall()]

    # SQL injection simulation
    attack_input = "ФеП-33' OR '1'='1"
    cursor.execute(query_sql, (attack_input,))
    injection_rows = [dict(r) for r in cursor.fetchall()]

    conn.close()

    return {
        "query_sql": query_sql,
        "target_group": target_group,
        "normal_rows": normal_rows,
        "attack_input": attack_input,
        "injection_rows": injection_rows,
        "injection_prevented": len(injection_rows) == 0,
    }
```

Нижче показав SQL та результати, які capture_report_evidence.py зберіг у JSON. Це перегляд раніше виконаної перевірки, не вигаданий термінальний сеанс.

Для читабельності розбив справжній збережений SQL на рядки, змінивши лише пробіли та переноси. Це форматування захопленого запиту, а не нове виконання.

```sql
SELECT s.id, s.first_name, s.last_name, g.name AS group_name,
    s.average_grade, s.email
FROM students s
JOIN groups g ON s.group_id = g.id
WHERE lower(g.name) = lower(?)
ORDER BY s.id;
```

id	Ім'я	Прізвище	Група	Середній бал
1	Тарас	Шевченко	ФеП-33	94.5
2	Леся	Українка	ФеП-33	96.0

Перевірка	Переданий параметр	Рядків
Звичайний пошук	ФеП-33	2
SQL injection	ФеП-33' OR '1'='1	0

9. Транзакція масового переведення та ACID

src/student_manager/persistence/service.py - StudentPersistenceService.__init__

```python
def __init__(
    self,
    session_factory: sessionmaker[Session] | None = None,
    session: Session | None = None,
) -> None:
    if (session_factory is None) == (session is None):
        raise ValueError("Provide exactly one of session_factory or session")
    self._session_factory = session_factory
    self._session = session
```

src/student_manager/persistence/service.py - StudentPersistenceService._write_session

```python
@contextmanager
def _write_session(self) -> Generator[Session, None, None]:
    # контекст сесії з фіксацією транзакції та відкатом при помилці
    if self._session_factory is not None:
        with self._session_factory() as s:
            try:
                s.begin()
                yield s
                s.commit()
            except Exception:
                if s.in_transaction():
                    s.rollback()
                raise
    elif self._session is not None:
        s = self._session
        if not s.in_transaction():
            try:
                s.begin()
                yield s
                s.commit()
            except Exception:
                if s.in_transaction():
                    s.rollback()
                raise
        else:
            with s.begin_nested():
                yield s
```

src/student_manager/persistence/service.py - StudentPersistenceService.transfer_students

```python
def transfer_students(
    self, student_ids: Sequence[int], target_group_id: int
) -> int:
    # масове атомарне переведення студентів до цільової групи
    _validate_positive_int(target_group_id, "target_group_id")
    if not student_ids:
        return 0

    # перевіряємо наявність дублікатів та валідність кожного id
    seen: set[int] = set()
    for sid in student_ids:
        _validate_positive_int(sid, "student_id")
        if sid in seen:
            raise ValueError(f"Duplicate student ID {sid} in transfer list")
        seen.add(sid)

    with self._write_session() as s:
        target_group = s.get(GroupRow, target_group_id)
        if target_group is None:
            raise ValueError(
                f"Target group with ID {target_group_id} does not exist"
            )

        for sid in student_ids:
            row = s.get(StudentRow, sid)
            if row is None:
                raise ValueError(f"Student with ID {sid} does not exist")
            row.group_id = target_group_id

        s.flush()
        return len(student_ids)
```

commit фіксує всю транзакцію, rollback скасовує незавершені зміни після помилки. Сервіс не виконує проміжний commit для кожного студента. Для наданої зовні активної Session використовує SAVEPOINT через begin_nested; остаточний commit залишається власнику зовнішньої транзакції.

Властивість ACID	**Реалізація та межі перевірки**
Atomicity	Масове переведення атомарне: помилка ID відкидає зміни; тест читає стан повторно.
Consistency	PK, FK, NOT NULL, CHECK бала 0-100 та валідація Repository зберігають інваріанти.
Isolation	SQLite керує блокуванням і транзакціями; окрема Session на операцію. Конкурентний стрес-тест тут не заявляю.
Durability	commit зберігає дані у SQLite-файл; повторне читання перевіряє видимість. Аварію живлення не моделював.

10. Міграції Alembic

```
[alembic]
script_location = student_manager.persistence:migrations
prepend_sys_path = src
path_separator = os
file_template = %%(rev)s_%%(slug)s
timezone = UTC
sqlalchemy.url = sqlite:///student_manager.db

[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARN
handlers = console
qualname =

[logger_sqlalchemy]
level = WARN
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers =
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
```

src/student_manager/persistence/migrations/env.py

```python
from __future__ import annotations

import os
from logging.config import fileConfig
from typing import Any

from alembic import context
from sqlalchemy import engine_from_config, event, pool, text

from student_manager.persistence.models import Base

config = context.config

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata


def get_url() -> str:
    # безпечне отримання url з підтримкою target_url або override через DATABASE_URL
    target = getattr(config, "attributes", {}).get("target_url")
    if target:
        return str(target)
    env_url = os.getenv("DATABASE_URL")
    if env_url:
        return env_url
    url = config.get_main_option("sqlalchemy.url")
    if url:
        return url
    return "sqlite:///student_manager.db"


def run_migrations_offline() -> None:
```

```python
    # запуск міграцій в офлайн режимі без підключення до бд
    url = get_url()
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
        render_as_batch=True,
    )

    with context.begin_transaction():
        context.run_migrations()


def run_migrations_online() -> None:
    # запуск міграцій онлайн з контролем sqlite foreign keys
    connectable = config.attributes.get("connection", None)

    if connectable is None:
        url = get_url()
        configuration = config.get_section(config.config_ini_section, {})
        configuration["sqlalchemy.url"] = url
        connectable = engine_from_config(
            configuration,
            prefix="sqlalchemy.",
            poolclass=pool.NullPool,
        )

        if connectable.dialect.name == "sqlite":

            @event.listens_for(connectable, "connect")
            def _enable_sqlite_fk(
                dbapi_connection: Any, connection_record: Any
            ) -> None:
```

```
                cursor = dbapi_connection.cursor()
                cursor.execute("PRAGMA foreign_keys=ON")
                cursor.close()

        with connectable.connect() as connection:
            if connection.dialect.name == "sqlite":
                connection.execute(text("PRAGMA foreign_keys=ON"))
            context.configure(
                connection=connection,
                target_metadata=target_metadata,
                render_as_batch=True,
            )

            with context.begin_transaction():
                context.run_migrations()
            connection.commit()
        connectable.dispose()
    else:
        if connectable.dialect.name == "sqlite":
            connectable.execute(text("PRAGMA foreign_keys=ON"))
        context.configure(
            connection=connectable,
            target_metadata=target_metadata,
            render_as_batch=True,
        )

        with context.begin_transaction():
            context.run_migrations()
        if hasattr(connectable, "commit"):
            connectable.commit()


if context.is_offline_mode():
    run_migrations_offline()
```

```
else:
    run_migrations_online()
```

src/student_manager/persistence/migrations/versions/0002_student_email.py

```python
"""add student email column

Revision ID: 0002_student_email
Revises: 0001_initial
Create Date: 2026-03-01 10:05:00.000000

"""

from __future__ import annotations

from collections.abc import Sequence

import sqlalchemy as sa
from alembic import op

# revision identifiers, used by Alembic.
revision: str = "0002_student_email"
down_revision: str | None = "0001_initial"
branch_labels: str | Sequence[str] | None = None
depends_on: str | Sequence[str] | None = None


def upgrade() -> None:
    # додавання опціонального стовпця email до таблиці students
    with op.batch_alter_table("students", schema=None) as batch_op:
        batch_op.add_column(sa.Column("email", sa.String(length=255), nullable=True))


def downgrade() -> None:
    # видалення стовпця email зі збереженням решти даних
    with op.batch_alter_table("students", schema=None) as batch_op:
        batch_op.drop_column("email")
```

src/student_manager/lab7_demo.py - apply_migrations

```python
def apply_migrations(db_path: Path | str) -> None:
    # програмне застосування міграцій alembic через ресурси пакета
    resolved_path = Path(db_path).resolve()
    db_url = f"sqlite:///{resolved_path.as_posix()}"
    cfg = Config()
    cfg.set_main_option("script_location", "student_manager.persistence:migrations")
    cfg.set_main_option("sqlalchemy.url", db_url)
    cfg.attributes["target_url"] = db_url
    command.upgrade(cfg, "head")
```

tests/test_migrations.py - test_migration_lifecycle_upgrade_downgrade_preserves_data

```python
def test_migration_lifecycle_upgrade_downgrade_preserves_data(
    alembic_config: Config, temp_db_path: Path
) -> None:
    # тестуємо повний цикл міграцій 0001 -> 0002 -> 0001 -> 0002 зі збереженням даних
    # 1. апгрейд до базової ревізії 0001_initial
    command.upgrade(alembic_config, "0001_initial")

    engine = create_engine(f"sqlite:///{temp_db_path.as_posix()}")
    try:
        inspector = inspect(engine)
        tables = inspector.get_table_names()
        assert "groups" in tables
        assert "students" in tables

        # перевіряємо відсутність колонки email у базовій ревізії
        student_cols = [c["name"] for c in inspector.get_columns("students")]
        assert "id" in student_cols
        assert "first_name" in student_cols
        assert "last_name" in student_cols
        assert "average_grade" in student_cols
        assert "group_id" in student_cols
        assert "email" not in student_cols

        # вставляємо початкові дані у версії 0001
        with engine.connect() as conn:
            conn.execute(text("PRAGMA foreign_keys = ON"))
            conn.execute(text("INSERT INTO groups (id, name) VALUES (1, 'ФеП-33')"))
            conn.execute(
                text(
                    "INSERT INTO students (id, first_name, last_name, average_grade, group_id) "
                    "VALUES (1, 'Тарас', 'Шевченко', 95.5, 1)"
                )
            )
            conn.commit()
```

```
    finally:
        engine.dispose()

    # 2. апгрейд до 0002_student_email
    command.upgrade(alembic_config, "0002_student_email")

    engine = create_engine(f"sqlite:///{temp_db_path.as_posix()}")
    try:
        inspector = inspect(engine)
        student_cols_v2 = [c["name"] for c in inspector.get_columns("students")]
        assert "email" in student_cols_v2

        # перевіряємо збереження існуючих даних та значення email=None
        with engine.connect() as conn:
            row = conn.execute(
                text(
                    "SELECT id, first_name, last_name, average_grade, group_id, email FROM students WHERE id = 1"
                )
            ).fetchone()
            assert row is not None
            assert row[0] == 1
            assert row[1] == "Тарас"
            assert row[2] == "Шевченко"
            assert row[3] == 95.5
            assert row[4] == 1
            assert row[5] is None

            # додаємо студента з email у версії 0002
            conn.execute(
                text(
                    "INSERT INTO students (id, first_name, last_name, average_grade, group_id, email) "
                    "VALUES (2, 'Леся', 'Українка', 98.0, 1, 'lesya@lnu.edu.ua')"
                )
            )
```

```
            conn.commit()
    finally:
        engine.dispose()

    # 3. даунгрейд назад до 0001_initial
    command.downgrade(alembic_config, "0001_initial")

    engine = create_engine(f"sqlite:///{temp_db_path.as_posix()}")
    try:
        inspector = inspect(engine)
        student_cols_v1_again = [c["name"] for c in inspector.get_columns("students")]
        assert "email" not in student_cols_v1_again

        # перевіряємо збереження записів після даунгрейду
        with engine.connect() as conn:
            rows = conn.execute(
                text(
                    "SELECT id, first_name, last_name, average_grade, group_id FROM students ORDER BY id"
                )
            ).fetchall()
            assert len(rows) == 2
            assert rows[0][:3] == (1, "Тарас", "Шевченко")
            assert rows[1][:3] == (2, "Леся", "Українка")
    finally:
        engine.dispose()

    # 4. повторний апгрейд до head (0002_student_email)
    command.upgrade(alembic_config, "head")

    engine = create_engine(f"sqlite:///{temp_db_path.as_posix()}")
    try:
        inspector = inspect(engine)
        assert "email" in [c["name"] for c in inspector.get_columns("students")]
```

```
        with engine.connect() as conn:
            count = conn.execute(text("SELECT count(*) FROM students")).scalar_one()
            assert count == 2
    finally:
        engine.dispose()
```

Переглянув збережену перевірку життєвого циклу міграцій. Ревізія 0001 створює groups і students, 0002 додає nullable email; downgrade повертає початкову схему зі збереженням записів.

Етап	Зафіксований результат
upgrade 0001	success
upgrade 0002	success
email після upgrade	True
downgrade 0001	True

Етап	Зафіксований результат
Рядків після downgrade	2
Відмінностей metadata на head	0

11. Демонстрація роботи програми

src/student_manager/lab7_demo.py - run_demo

```python
def run_demo(
    database: str | Path,
    *,
    migrations: bool = True,
    refuse_existing: bool = True,
) -> dict[str, Any]:
    # запуск повного демонстраційного сценарію роботи з бд
    db_path = Path(database).resolve()
    if refuse_existing and db_path.exists():
        raise FileExistsError(
            f"Файл бази даних '{db_path}' вже існує. Перезапис заборонено."
        )

    # застосовуємо міграції alembic до чистої бд
    if migrations:
        apply_migrations(db_path)

    db_url = f"sqlite:///{db_path.as_posix()}"
    engine = create_database_engine(db_url)
    session_factory = create_session_factory(engine)
    service = StudentPersistenceService(session_factory=session_factory)

    try:
        # 1. створення груп
        grp_fep33 = service.create_group("ФеП-33")
        grp_fep34 = service.create_group("ФеП-34")

        # 2. додавання трьох фіктивних студентів
        s1 = service.create_student(
            Student("Тарас", "Шевченко", "ФеП-33", 94.5, student_id=1),
            group_id=grp_fep33.id,
            email="shevchenko@lnu.edu.ua",
        )
        s2 = service.create_student(
```

```
        Student("Леся", "Українка", "ФеП-33", 96.0, student_id=2),
        group_id=grp_fep33.id,
        email="ukrainka@lnu.edu.ua",
    )
    s3 = service.create_student(
        Student("Іван", "Франко", "ФеП-34", 91.0, student_id=3),
        group_id=grp_fep34.id,
        email="franko@lnu.edu.ua",
    )

    # 3. отримання за id та пагінований список
    fetched_s1 = service.get_student(s1.student_id or 1)
    all_students = service.list_students(offset=0, limit=100)

    # 4. пошук за прізвищем та фільтрація за групою
    by_surname = service.search_students_by_last_name("Шевченко")
    by_group = service.filter_students_by_group(grp_fep33.id)

    # 5. аналітика: найкращий студент та середній бал групи
    best_student = service.get_best_student()
    group_avg = service.get_group_average(grp_fep33.id)

    # 6. оновлення оцінки
    updated_s3 = service.update_student(s3.student_id or 3, average_grade=93.5)

    # 7. переведення студента
    transferred_count = service.transfer_students(
        [s3.student_id or 3], grp_fep33.id
    )

    # 8. демонстрація відкату транзакції при недійсному id
    s1_before = service.get_student(s1.student_id or 1)
    group_before_rollback = s1_before.group if s1_before else None
    rollback_success = False
```

```
    try:
        # передаємо валідний id разом із неіснуючим
        service.transfer_students([s1.student_id or 1, 9999], grp_fep34.id)
    except ValueError:
        rollback_success = True

    # перевіряємо свіжим читанням що стан студента 1 не змінився
    s1_after = service.get_student(s1.student_id or 1)
    assert s1_after is not None
    assert s1_after.group == group_before_rollback

    # 9. видалення студента та підрахунок залишку
    deleted = service.delete_student(s3.student_id or 3)
    remaining_count = service.count_students()

    return {
        "database": str(database),
        "groups": [
            {"id": grp_fep33.id, "name": grp_fep33.name},
            {"id": grp_fep34.id, "name": grp_fep34.name},
        ],
        "students_created": [
            {
                "student_id": s1.student_id,
                "full_name": s1.full_name,
                "group": s1.group,
                "average_grade": s1.average_grade,
            },
            {
                "student_id": s2.student_id,
                "full_name": s2.full_name,
                "group": s2.group,
                "average_grade": s2.average_grade,
            },
```

```
                {
                    "student_id": s3.student_id,
                    "full_name": s3.full_name,
                    "group": s3.group,
                    "average_grade": s3.average_grade,
                },
            ],
            "student_fetched": {
                "student_id": fetched_s1.student_id if fetched_s1 else None,
                "full_name": fetched_s1.full_name if fetched_s1 else None,
                "group": fetched_s1.group if fetched_s1 else None,
            },
            "total_listed": len(all_students),
            "surname_search": [
                {"student_id": s.student_id, "full_name": s.full_name}
                for s in by_surname
            ],
            "group_filter": [
                {"student_id": s.student_id, "full_name": s.full_name} for s in by_group
            ],
            "best_student": {
                "full_name": best_student.full_name if best_student else None,
                "average_grade": best_student.average_grade if best_student else None,
            },
            "group_average": group_avg,
            "grade_updated": {
                "student_id": updated_s3.student_id if updated_s3 else None,
                "average_grade": updated_s3.average_grade if updated_s3 else None,
            },
            "transfer_success": transferred_count == 1,
            "rollback_demonstrated": rollback_success,
            "deleted": deleted,
            "remaining_count": remaining_count,
        }
```

```
    finally:
        # закриваємо пул з'єднань рушія
        engine.dispose()
```

src/student_manager/lab7_demo.py - main

```
def main(argv: list[str] | None = None) -> int:
    # точка входу командного рядка
    parser = argparse.ArgumentParser(
        description="Демонстрація persistence layer лабораторної роботи 7"
    )
    parser.add_argument(
        "--database",
        required=True,
        type=str,
        help="Шлях до нового файлу бази даних SQLite",
    )
    parser.add_argument(
        "--migrations",
        action="store_true",
        default=True,
        help="Застосувати міграції alembic перед виконанням",
    )

    args = parser.parse_args(argv)

    try:
        data = run_demo(
            database=args.database,
            migrations=args.migrations,
            refuse_existing=True,
        )

        # лаконічний вивід результатів українською мовою
        print(f"[Міграції] Застосовано alembic head для {data['database']}")
        print(
            f"[Групи] Створено: {data['groups'][0]['id']} ({data['groups'][0]['name']}), "
            f"{data['groups'][1]['id']} ({data['groups'][1]['name']})"
        )
        print("[Студенти] Додано:")
```

```
        for s in data["students_created"]:
            print(
                f"- {s['student_id']}: {s['full_name']}, група {s['group']}, "
                f"сер. бал: {s['average_grade']:.2f}"
            )
        print(
            f"[Отримання] Студент {data['student_fetched']['student_id']}: "
            f"{data['student_fetched']['full_name']} ({data['student_fetched']['group']})"
        )
        print(f"[Список] Всього студентів: {data['total_listed']}")
        print(
            f"[Пошук] За прізвищем 'Шевченко': {len(data['surname_search'])} запис(ів)"
        )
        print(f"[Фільтр] Студенти групи ФеП-33: {len(data['group_filter'])} запис(ів)")
        print(
            f"[Аналітика] Найкращий студент: {data['best_student']['full_name']} "
            f"({data['best_student']['average_grade']:.2f})"
        )
        if data["group_average"] is not None:
            print(f"[Аналітика] Середній бал групи ФеП-33: {data['group_average']:.2f}")
        print(
            f"[Оновлення] Студента {data['grade_updated']['student_id']} оновлено бал: "
            f"{data['grade_updated']['average_grade']:.2f}"
        )
        print(
            f"[Переведення] Переведення студента в нову групу: "
            f"{'успішно' if data['transfer_success'] else 'невдало'}"
        )
        print(
            "[Транзакція] Спроба масового переводу з недійсним ID (9999): помилку перехоплено"
        )
        print(
            f"[Атомарність] Стан студента не змінився після збою: "
            f"{'відкат підтверджено' if data['rollback_demonstrated'] else 'помилка'}"
        )
```

```
        )
        print(
            f"[Видалення] Видалення студента: {data['deleted']}, "
            f"залишилось студентів: {data['remaining_count']}"
        )
        return 0
    except (RuntimeError, ValueError, TypeError, FileExistsError, OSError) as exc:
        sys.stderr.write(f"Помилка виконання: {exc}\n")
        return 1
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m student_manager.main --lab7 F:/university/3yr-
1/professional_python/.local_workflow/lab7/report_assets/evidence_demo.db
[Міграції] Застосовано alembic head для F:/university/3yr-
1/professional_python/.local_workflow/lab7/report_assets/evidence_demo.db
[Групи] Створено: 1 (ФеП-33), 2 (ФеП-34)
[Студенти] Додано:
- 1: Тарас Шевченко, група ФеП-33, сер. бал: 94.50
- 2: Леся Українка, група ФеП-33, сер. бал: 96.00
- 3: Іван Франко, група ФеП-34, сер. бал: 91.00
[Отримання] Студент 1: Тарас Шевченко (ФеП-33)
[Список] Всього студентів: 3
[Пошук] За прізвищем 'Шевченко': 1 запис(ів)
[Фільтр] Студенти групи ФеП-33: 2 запис(ів)
[Аналітика] Найкращий студент: Леся Українка (96.00)
[Аналітика] Середній бал групи ФеП-33: 95.25
[Оновлення] Студента 3 оновлено бал: 93.50
[Переведення] Переведення студента в нову групу: успішно
[Транзакція] Спроба масового переводу з недійсним ID (9999): помилку перехоплено
[Атомарність] Стан студента не змінився після збою: відкат підтверджено
[Видалення] Видалення студента: True, залишилось студентів: 2
```

12. Integration tests та справжній pytest -v

tests/test_persistence.py - temp_db_path

```python
@pytest.fixture
def temp_db_path() -> Generator[Path, None, None]:
    # тимчасовий ізольований sqlite файл для кожного тесту
    with tempfile.NamedTemporaryFile(suffix=".db", delete=False) as f:
        path = Path(f.name)
    yield path
    if path.exists():
        path.unlink(missing_ok=True)
```

tests/test_persistence.py - engine

```python
@pytest.fixture
def engine(temp_db_path: Path) -> Generator[Engine, None, None]:
    # створюємо рушій бд з обов'язковим увімкненням fk
    eng = create_database_engine(f"sqlite:///{temp_db_path}")
    Base.metadata.create_all(bind=eng)
    yield eng
    eng.dispose()
```

tests/test_persistence.py - session_factory

```python
@pytest.fixture
def session_factory(engine: Engine) -> sessionmaker[Session]:
    # фабрика сесій для тестів
    return create_session_factory(engine)
```

tests/test_persistence.py - test_sqlite_foreign_keys_enforced

```python
def test_sqlite_foreign_keys_enforced(engine: Engine) -> None:
    # перевіряємо що pragma foreign_keys реально активна
    with engine.connect() as conn:
        res = conn.execute(text("PRAGMA foreign_keys")).scalar()
        assert res == 1
```

tests/test_persistence.py - test_student_crud_and_validation

```python
def test_student_crud_and_validation(
    session_factory: sessionmaker[Session],
) -> None:
    # перевірка збереження, читання, оновлення і видалення студента
    with session_scope(session_factory) as session:
        g_repo = GroupRepository(session)
        group = g_repo.create("ФеП-33")
        s_repo = StudentRepository(session)

        st = Student("Іван", "Франко", "ФеП-33", 92.5)
        created = s_repo.create(st, group_id=group.id, email="franko@lnu.edu.ua")
        assert created.student_id == 1
        assert created.first_name == "Іван"
        assert created.last_name == "Франко"

    with session_scope(session_factory) as session:
        s_repo = StudentRepository(session)
        loaded = s_repo.get(1)
        assert loaded is not None
        assert loaded.student_id == 1
        assert loaded.last_name == "Франко"
        assert loaded.group == "ФеП-33"

        dto = s_repo.get_dto(1)
        assert dto is not None
        assert isinstance(dto, StudentDTO)
        assert dto.email == "franko@lnu.edu.ua"

        # оновлення студента
        updated = s_repo.update(1, average_grade=98.0, email="new_franko@lnu.edu.ua")
        assert updated is not None
        assert updated.average_grade == 98.0

    with session_scope(session_factory) as session:
```

```python
        s_repo = StudentRepository(session)
        after_update = s_repo.get_dto(1)
        assert after_update is not None
        assert after_update.average_grade == 98.0
        assert after_update.email == "new_franko@lnu.edu.ua"

        # видалення студента
        deleted = s_repo.delete(1)
        assert deleted is True
        assert s_repo.get(1) is None
```

tests/test_persistence.py - test_group_crud_and_constraints

```python
def test_group_crud_and_constraints(
    session_factory: sessionmaker[Session],
) -> None:
    # перевірка crud операцій та обмежень груп
    with session_scope(session_factory) as session:
        repo = GroupRepository(session)
        g1 = repo.create("ФеП-33")
        assert isinstance(g1, GroupDTO)
        assert g1.id == 1
        assert g1.name == "ФеП-33"

    with session_scope(session_factory) as session:
        repo = GroupRepository(session)
        fetched = repo.get(1)
        assert fetched is not None
        assert fetched.name == "ФеП-33"

        groups = repo.list()
        assert len(groups) == 1
        assert groups[0].name == "ФеП-33"

    # заборона порожньої назви
    with session_scope(session_factory) as session:
        repo = GroupRepository(session)
        with pytest.raises(ValueError):
            repo.create("   ")

    # заборона дублікатів назви групи (unique constraint)
    with pytest.raises(IntegrityError), session_scope(session_factory) as session:
        repo = GroupRepository(session)
        repo.create("ФеП-33")

    # успішне видалення порожньої групи
    with session_scope(session_factory) as session:
```

```python
        repo = GroupRepository(session)
        g2 = repo.create("ФеП-34")
        assert repo.delete(g2.id) is True
        assert repo.get(g2.id) is None
```

tests/test_persistence.py - test_student_queries_and_pagination

```python
def test_student_queries_and_pagination(
    session_factory: sessionmaker[Session],
) -> None:
    # перевірка пошуку за прізвищем, фільтрації та пагінації
    with session_scope(session_factory) as session:
        g_repo = GroupRepository(session)
        g1 = g_repo.create("ФеП-33")
        g2 = g_repo.create("ФеП-34")
        s_repo = StudentRepository(session)

        s_repo.create(Student("Ольга", "Коваль", "ФеП-33", 88.0), group_id=g1.id)
        s_repo.create(Student("Микола", "Мельник", "ФеП-33", 94.0), group_id=g1.id)
        s_repo.create(Student("Ірина", "коваль", "ФеП-34", 91.0), group_id=g2.id)

    with session_scope(session_factory) as session:
        s_repo = StudentRepository(session)

        # пошук за прізвищем незалежно від регістру
        kovals = s_repo.search_by_last_name("  КОВАЛЬ  ")
        assert len(kovals) == 2
        assert {s.student_id for s in kovals} == {1, 3}

        # фільтрація за групою
        fep33_students = s_repo.filter_by_group(g1.id)
        assert len(fep33_students) == 2
        assert fep33_students[0].first_name == "Ольга"

        # пагінація
        page1 = s_repo.list(offset=0, limit=2)
        assert len(page1) == 2
        page2 = s_repo.list(offset=2, limit=2)
        assert len(page2) == 1
        assert page2[0].student_id == 3
```

tests/test_persistence.py - test_student_aggregates_and_deterministic_ties

```python
def test_student_aggregates_and_deterministic_ties(
    session_factory: sessionmaker[Session],
) -> None:
    # перевірка агрегацій: середній бал, найкращий студент, tie-breaker
    with session_scope(session_factory) as session:
        g_repo = GroupRepository(session)
        g1 = g_repo.create("ФеП-33")
        g_repo.create("ФеП-34")
        s_repo = StudentRepository(session)

        # для порожньої групи агрегати повертають None
        assert s_repo.average(g1.id) is None
        assert s_repo.best(g1.id) is None
        assert s_repo.count(g1.id) == 0

        # додаємо студентів з однаковим найвищим балом для перевірки детермінізму
        s_repo.create(Student("Анна", "Білик", "ФеП-33", 95.0), group_id=g1.id)  # id 1
        s_repo.create(Student("Богдан", "Вовк", "ФеП-33", 95.0), group_id=g1.id)  # id 2
        s_repo.create(Student("Віктор", "Гай", "ФеП-33", 80.0), group_id=g1.id)  # id 3

        avg = s_repo.average(g1.id)
        assert avg is not None
        assert round(avg, 2) == 90.0

        assert s_repo.count(g1.id) == 3

        # детермінований вибір найкращого при однаковому балі (за id asc)
        best_student = s_repo.best(g1.id)
        assert best_student is not None
        assert best_student.student_id == 1
        assert best_student.average_grade == 95.0
```

tests/test_persistence.py - test_service_rejects_session_and_factory_together

```python
def test_service_rejects_session_and_factory_together(
    session_factory: sessionmaker[Session],
) -> None:
    with (
        session_factory() as session,
        pytest.raises(ValueError, match="session_factory or session"),
    ):
        StudentPersistenceService(session_factory=session_factory, session=session)
```

tests/test_persistence.py - test_service_successful_bulk_transfer

```python
def test_service_successful_bulk_transfer(
    session_factory: sessionmaker[Session],
) -> None:
    # успішне масове переведення студентів між групами через сервіс
    service = StudentPersistenceService(session_factory=session_factory)
    g1 = service.create_group("ФеП-33")
    g2 = service.create_group("ФеП-34")

    s1 = service.create_student(Student("Студент1", "Тест", "ФеП-33", 80.0), g1.id)
    s2 = service.create_student(Student("Студент2", "Тест", "ФеП-33", 85.0), g1.id)
    assert s1.student_id is not None
    assert s2.student_id is not None

    count = service.transfer_students([s1.student_id, s2.student_id], g2.id)
    assert count == 2

    # перевіряємо що студенти тепер у новій групі
    in_g2 = service.filter_students_by_group(g2.id)
    assert len(in_g2) == 2
    assert in_g2[0].group == "ФеП-34"
    assert in_g2[1].group == "ФеП-34"

    in_g1 = service.filter_students_by_group(g1.id)
    assert len(in_g1) == 0
```

tests/test_persistence.py - test_service_bulk_transfer_atomic_rollback_on_failure

```python
def test_service_bulk_transfer_atomic_rollback_on_failure(
    session_factory: sessionmaker[Session],
) -> None:
    # перевірка атомарного відкату при збої всередині переведення
    service = StudentPersistenceService(session_factory=session_factory)
    g1 = service.create_group("ФеП-33")
    g2 = service.create_group("ФеП-34")

    s1 = service.create_student(Student("Студент1", "Тест", "ФеП-33", 80.0), g1.id)
    assert s1.student_id is not None

    # передаємо валідний s1 і неіснуючий id 9999
    with pytest.raises(ValueError):
        service.transfer_students([s1.student_id, 9999], g2.id)

    # s1 не повинен бути переведений - транзакція відкотилась повністю
    after_failed = service.get_student(s1.student_id)
    assert after_failed is not None
    assert after_failed.group == "ФеП-33"

    # група ФеП-34 залишається порожньою
    assert len(service.filter_students_by_group(g2.id)) == 0
```

tests/test_persistence_atomic_validation.py - test_invalid_repository_update_does_not_leave_partial_mutation

```python
@pytest.mark.parametrize(
    "changes",
    [
        {"first_name": "Марія", "average_grade": 150.0},
        {"first_name": "Марія", "last_name": " "},
        {"first_name": "Марія", "group_id": 999},
        {"first_name": "Марія", "email": 123},
    ],
)
def test_invalid_repository_update_does_not_leave_partial_mutation(tmp_path, changes):
    engine = create_database_engine(f"sqlite:///{tmp_path / 'atomic.sqlite'}")
    try:
        Base.metadata.create_all(engine)
        with Session(engine) as session:
            group = GroupRepository(session).create("ФеП-33")
            student = StudentRepository(session).create(
                Student("Олена", "Коваль", group.name, 90.0), group.id
            )
            session.commit()
            assert student.student_id is not None
            repository = StudentRepository(session)
            with pytest.raises((ValueError, TypeError)):
                repository.update(student.student_id, **changes)
            session.commit()
            actual = repository.get(student.student_id)
            assert actual is not None
            assert actual.first_name == "Олена"
            assert actual.average_grade == 90.0
    finally:
        engine.dispose()
```

tests/test_dbapi.py - test_find_students_by_group_sql_injection_safe

```python
def test_find_students_by_group_sql_injection_safe(temp_db_path: Path) -> None:
    # параметризація захищає від ін'єкцій у запиті
    injection_query = "' OR '1'='1"
    students = find_students_by_group(temp_db_path, injection_query)
    assert students == []

    drop_query = "ФеП-33'; DROP TABLE students; --"
    students = find_students_by_group(temp_db_path, drop_query)
    assert students == []

    # перевіряємо що таблиця не дропнулась
    valid_students = find_students_by_group(temp_db_path, "ФеП-33")
    assert len(valid_students) == 2
```

tests/test_migration_target.py - test_explicit_demo_path_is_not_overridden_by_database_environment

```python
def test_explicit_demo_path_is_not_overridden_by_database_environment(
    tmp_path, monkeypatch
):
    requested = tmp_path / "requested.sqlite"
    unrelated = tmp_path / "unrelated.sqlite"
    monkeypatch.setenv("DATABASE_URL", f"sqlite:///{unrelated.as_posix()}")
    apply_migrations(requested)
    assert requested.exists()
    assert not unrelated.exists()
```

Нижче навів повний збережений запуск pytest -v: 40 успішних тестів. Код вище читаю з поточного дерева, а цей transcript належить моменту захоплення; це не повторний запуск після подальших змін.

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest tests/test_persistence.py
tests/test_persistence_atomic_validation.py tests/test_migrations.py tests/test_dbapi.py
tests/test_lab7_demo.py tests/test_lab7_cli.py tests/test_migration_target.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collecting ... collected 40 items

tests/test_persistence.py::test_sqlite_foreign_keys_enforced PASSED      [  2%]
tests/test_persistence.py::test_session_scope_commit_and_rollback PASSED [  5%]
tests/test_persistence.py::test_group_crud_and_constraints PASSED        [  7%]
tests/test_persistence.py::test_group_deletion_restricted_when_populated PASSED [ 10%]
tests/test_persistence.py::test_student_crud_and_validation PASSED       [ 12%]
tests/test_persistence.py::test_student_identifier_rejections PASSED     [ 15%]
tests/test_persistence.py::test_student_check_constraints PASSED         [ 17%]
tests/test_persistence.py::test_student_queries_and_pagination PASSED    [ 20%]
tests/test_persistence.py::test_student_aggregates_and_deterministic_ties PASSED [ 22%]
tests/test_persistence.py::test_two_students_with_same_name_allowed PASSED [ 25%]
tests/test_persistence.py::test_service_rejects_session_and_factory_together PASSED [ 27%]
```

Продовження того самого stdout, фрагмент 2

```
tests/test_persistence.py::test_service_successful_bulk_transfer PASSED  [ 30%]
tests/test_persistence.py::test_service_bulk_transfer_atomic_rollback_on_failure PASSED [ 32%]
tests/test_persistence.py::test_service_bulk_transfer_duplicate_ids_rejected PASSED [ 35%]
tests/test_persistence.py::test_detached_dto_access_after_session_closed PASSED [ 37%]
tests/test_persistence_atomic_validation.py::test_invalid_repository_update_does_not_leave_partial_mutation
[changes0] PASSED [ 40%]
tests/test_persistence_atomic_validation.py::test_invalid_repository_update_does_not_leave_partial_mutation
[changes1] PASSED [ 42%]
tests/test_persistence_atomic_validation.py::test_invalid_repository_update_does_not_leave_partial_mutation
[changes2] PASSED [ 45%]
tests/test_persistence_atomic_validation.py::test_invalid_repository_update_does_not_leave_partial_mutation
[changes3] PASSED [ 47%]
tests/test_migrations.py::test_migration_lifecycle_upgrade_downgrade_preserves_data PASSED [ 50%]
tests/test_migrations.py::test_schema_constraints_and_foreign_keys PASSED [ 52%]
tests/test_migrations.py::test_schema_matches_orm_metadata_at_head PASSED [ 55%]
tests/test_migrations.py::test_env_database_url_override PASSED          [ 57%]
tests/test_dbapi.py::test_find_students_by_group_exact PASSED            [ 60%]
tests/test_dbapi.py::test_find_students_by_group_unicode_casefold PASSED [ 62%]
tests/test_dbapi.py::test_find_students_by_group_sql_injection_safe PASSED [ 65%]
tests/test_dbapi.py::test_find_students_by_group_special_characters PASSED [ 67%]
tests/test_dbapi.py::test_find_students_by_group_nonexistent_or_empty PASSED [ 70%]
tests/test_dbapi.py::test_find_students_by_group_invalid_type PASSED     [ 72%]
tests/test_dbapi.py::test_find_students_by_group_closes_connection PASSED [ 75%]
tests/test_dbapi.py::test_find_students_by_group_closes_on_query_error PASSED [ 77%]
tests/test_dbapi.py::test_get_connection_enforces_foreign_keys_and_casefold PASSED [ 80%]
```

Продовження того самого stdout, фрагмент 3

```
tests/test_lab7_demo.py::test_run_demo_creates_data_and_returns_dict PASSED [ 82%]
tests/test_lab7_demo.py::test_run_demo_refuses_existing_file PASSED      [ 85%]
tests/test_lab7_demo.py::test_main_cli_success PASSED                    [ 87%]
tests/test_lab7_demo.py::test_main_cli_refuses_existing_file PASSED      [ 90%]
tests/test_lab7_demo.py::test_main_cli_missing_database_arg PASSED       [ 92%]
tests/test_lab7_demo.py::test_atomic_rollback_verification PASSED        [ 95%]
tests/test_lab7_cli.py::test_lab7_mode_uses_existing_entrypoint PASSED   [ 97%]
tests/test_migration_target.py::test_explicit_demo_path_is_not_overridden_by_database_environment PASSED
[100%]

============================== 40 passed in 8.46s ==============================
```

13. Письмові відповіді на контрольні запитання

1. Що таке база даних?

База даних - це організована сукупність структурованих даних, збережених в електронному вигляді, якою керує спеціалізована система управління базами даних (СУБД). Вона забезпечує ефективне зберігання, швидкий пошук, оновлення та цілісність інформації.

2. Що таке реляційна база даних?

Реляційна база даних - це база даних, побудована на реляційній моделі, в якій дані організовані у вигляді пов'язаних між собою таблиць із фіксованими стовпцями та рядками. Зв'язки між записами різних таблиць встановлюються за допомогою первинних і зовнішніх ключів.

3. Що таке таблиця?

Таблиця - це основний структурний елемент реляційної бази даних, призначений для зберігання сутностей одного типу у вигляді двовимірної сітки. Вона складається з іменованих стовпців із визначеними типами даних та рядків із конкретними записами.

4. Що таке рядок таблиці?

Рядок таблиці (або кортеж, запис) представляє один конкретний екземпляр сутності, що містить повний набір значень для всіх стовпців цієї таблиці. Кожен рядок зазвичай однозначно ідентифікується значенням первинного ключа.

5. Що таке поле?

Поле (або стовпець, атрибут) - це елемент структури таблиці, що описує одну окрему характеристику сутності з фіксованим типом даних. На перетині рядка та стовпця зберігається конкретне значення цієї характеристики для відповідного запису.

6. Що таке SQL?

SQL (Structured Query Language) - це стандартизована декларативна мова програмування, призначена для опису, маніпулювання, вибірки та адміністрування даних у реляційних СУБД. За допомогою команд SQL створюються структури бази, додаються, змінюються та фільтруються необхідні записи.

7. Для чого використовується CREATE TABLE?

Команда CREATE TABLE використовується для створення нової таблиці в базі даних із заданою структурою. Вона визначає назви й типи даних колонок, а також обмеження цілісності, такі як первинні ключі, зовнішні ключі, обмеження NOT NULL та індекси.

8. Що означає CRUD?

CRUD - це акронім для чотирьох базових операцій маніпулювання даними в сховищі: Create (створення), Read (читання), Update (оновлення) та Delete (видалення). Цей набір операцій становить основу взаємодії з будь-якими персистентними сутностями.

9. Яка SQL-команда відповідає Create?

Операції Create у SQL відповідає команда INSERT INTO, яка додає один або кілька нових рядків із зазначеними значеннями полів до таблиці.

10. Яка команда відповідає Read?

Операції Read у SQL відповідає команда SELECT, яка здійснює пошук, фільтрацію та вибірку даних із однієї або кількох таблиць бази.

11. Яка команда відповідає Update?

Операції Update у SQL відповідає команда UPDATE, яка змінює значення вказаних стовпців в існуючих рядках таблиці згідно з умовою фільтрації WHERE.

12. Яка команда відповідає Delete?

Операції Delete у SQL відповідає команда DELETE FROM, яка видаляє з таблиці рядки, що задовольняють критеріям вибірки в блоці WHERE.

13. Що таке SQLite?

SQLite - це легка, вбудована реляційна СУБД, яка зберігає всю базу даних в одному файлі на диску або безпосередньо в оперативній пам'яті. Вона працює як бібліотека всередині процесу застосунку і не потребує окремого сервера чи складного адміністрування.

14. Які переваги SQLite?

Основними перевагами SQLite є відсутність потреби в конфігурації сервера (serverless), нульове адміністрування, швидкість виконання та повна підтримка транзакцій за стандартом ACID. Також вона ідеально підходить для локального тестування завдяки підтримці баз даних у пам'яті (:memory:).

15. Що таке Python DB-API?

Python DB-API (PEP 249) - це загальноприйнята специфікація інтерфейсу для роботи з базами даних у мові Python. Вона стандартизує методи підключення (Connection), виконання запитів через курсори (Cursor) та обробку результатів і винятків для різних рушіїв СУБД.

16. Для чого використовується Connection?

Об'єкт Connection представляє активний сеанс зв'язку між клієнтським застосунком і базою даних. Він використовується для створення курсорів, а також для керування межами транзакцій через методи commit() та rollback().

17. Для чого використовується Cursor?

Об'єкт Cursor використовується для надсилання та виконання SQL-запитів до бази даних, а також для вибірки отриманих результатів. За допомогою методів execute(), executemany(), fetchone() і fetchall() курсор надає доступ до результуючого набору рядків.

18. Для чого потрібен commit()?

Метод commit() фіксує всі зміни поточного транзакційного блоку, назавжди зберігаючи їх у базі даних і роблячи видимими для інших з'єднань.

19. Для чого потрібен rollback()?

Метод Connection.rollback() або Session.rollback() скасовує незавершену поточну транзакцію, а не автоматично повертає до останнього SAVEPOINT. Для часткового відкату використовують ROLLBACK TO SAVEPOINT або rollback вкладеної транзакції. У моєму сервісі власна транзакція відкочується повністю після помилки, а для вже активної зовнішньої Session контекст begin_nested() відкочує свій SAVEPOINT; остаточне рішення щодо зовнішньої транзакції залишається її власнику.

20. Що таке SQL injection?

SQL-ін'єкція - це вразливість безпеки, при якій зловмисник впроваджує довільний SQL-код у запит через неперевірені вхідні параметри користувача. Це може призвести до несанкціонованого читання, модифікації або повного знищення даних у базі.

21. Що таке parameterized query?

Параметризований запит передає SQL-шаблон і значення окремо: драйвер зв'язує значення з плейсхолдерами, наприклад ? у sqlite3, тому вони не стають SQL-кодом. Такий захист стосується переданих значень, але не довільних назв таблиць, колонок чи напрямку сортування. Динамічні ідентифікатори та інші структурні частини запиту потрібно обирати з дозволеного списку або будувати засобами SQLAlchemy, а не підставляти неперевірений ввід.

22. Чому не рекомендується формувати SQL через f-string?

Підстановка неперевірених значень через f-string змішує дані із SQL-кодом і може спричинити SQL injection та помилки з лапками чи типами. Для значень використовую параметри, а для динамічних ідентифікаторів - дозволений список. Стабільний текст параметризованого запиту також сприяє повторному

використанню підготовлених команд або планів, залежно від драйвера та СУБД; f-string із різними літералами може погіршувати кешування, але не робить його принципово неможливим.

23. Що таке Primary Key?

Primary Key (первинний ключ) - це стовпець або набір стовпців таблиці, значення яких гарантовано унікальні та не містять NULL, однозначно ідентифікуючи кожен рядок. Він слугує базовою опорою для побудови індексів та організації зв'язків між таблицями.

24. Що таке Foreign Key?

Foreign Key (зовнішній ключ) - це стовпець таблиці, який посилається на первинний ключ іншої таблиці для створення взаємозв'язку між сутностями. Він забезпечує реляційну цілісність, забороняючи додавання записів з посиланнями на неіснуючі батьківські елементи.

25. Що таке транзакція?

Транзакція - це послідовність операцій з базою даних, яка виконується як неподільна логічна одиниця роботи за принципом "все або нічого". Якщо хоча б одна дія завершується збоєм, уся транзакція відкочується, гарантуючи збереження цілісності даних.

26. Що означає Atomicity?

Atomicity (атомарність) гарантує, що всі операції всередині транзакції будуть виконані повністю та зафіксовані, або жодна з них не залишить змін у базі даних у разі помилки.

27. Що означає Consistency?

Consistency (узгодженість) гарантує, що транзакція переводить базу даних виключно з одного коректного стану в інший коректний стан, дотримуючись усіх визначених обмежень цілісності та бізнес-правил.

28. Що означає Isolation?

Isolation (ізольованість) гарантує, що паралельні транзакції не впливають одна на одну, а проміжні результати незавершеної транзакції залишаються недоступними для інших сесій до моменту фіксації.

29. Що означає Durability?

Durability (довговічність) гарантує, що після успішної фіксації транзакції внесені зміни надійно зберігаються на диску і не будуть втрачені навіть при раптовому апаратному чи системному збої.

30. Що означає ACID?

ACID - це абревіатура чотирьох фундаментальних властивостей надійної транзакційної системи: Atomicity (атомарність), Consistency (узгодженість), Isolation (ізольованість) та Durability (довговічність). Дотримання вимог ACID гарантує коректність і передбачуваність роботи бази даних при збоях та одночасному доступі.

31. Що таке persistence layer?

Persistence layer (рівень збереження даних) - це архітектурний рівень застосунку, відповідальний за взаємодію зі сховищем і збереження стану об'єктів між сеансами роботи програми. Він інкапсулює низькорівневу роботу з базою даних, відокремлюючи її від бізнес-логіки.

32. Для чого потрібний Repository Pattern?

Шаблон Repository інкапсулює логіку доступу до бази даних і надає бізнес-рівню об'єктно-орієнтований інтерфейс, схожий на роботу зі звичайною колекцією в пам'яті. Це ізолює доменний шар від деталей СУБД та суттєво спрощує модульне тестування.

33. Чому Repository відокремлюють від Service Layer?

Відокремлення Repository від Service Layer реалізує принцип єдиної відповідальності: сервіси містять бізнес-правила та оркеструють операції, тоді як репозиторії займаються виключно читанням і збереженням сутностей. Завдяки цьому бізнес-логіку можна легко тестувати за допомогою моків або тестових репозиторіїв.

34. Що таке ORM?

ORM (Object-Relational Mapping) - це технологія, яка автоматично відображає реляційні таблиці бази даних на об'єктно-орієнтовані класи мови програмування та навпаки. Вона дозволяє розробникам взаємодіяти з даними через звичні атрибути й методи об'єктів без написання рутинного SQL-коду.

35. Для чого використовується SQLAlchemy?

SQLAlchemy використовується як повнофункціональний інструментарій для взаємодії з реляційними базами даних у Python, що поєднує низькорівневий SQL Expression Language (Core) та високорівневий ORM. Вона керує пулом з'єднань, життєвим циклом сесій (Unit of Work) та транзакціями, забезпечуючи високу продуктивність і гнучкість.

36. Що таке ORM model?

ORM-модель - це Python-клас, успадкований від декларативної бази SQLAlchemy, що описує схему таблиці за допомогою атрибутів-колонок. Екземпляр такої моделі відповідає конкретному рядку в таблиці бази даних.

37. Що таке relationship()?

Функція relationship() у SQLAlchemy описує високорівневий зв'язок між ORM-моделями на основі зовнішніх ключів і забезпечує доступ до пов'язаних об'єктів як до звичайних атрибутів або списків. Вона також підтримує налаштування стратегій завантаження (lazy, joined, selectin) та каскадних операцій.

38. Що означає one-to-many?

Зв'язок one-to-many (один до багатьох) вказує на те, що один запис батьківської сутності може бути асоційований з багатьма записами дочірньої сутності, але кожен дочірній запис посилається лише на одного батька. У реляційній моделі це забезпечується розміщенням зовнішнього ключа в таблиці дочірніх сутностей.

39. Що означає many-to-many?

Зв'язок many-to-many (багато до багатьох) означає, що кілька екземплярів однієї сутності можуть бути пов'язані з кількома екземплярами іншої сутності. Для його представлення в реляційній базі використовується проміжна зв'язуюча таблиця (association table), яка містить зовнішні ключі на обидві цільові таблиці.

40. Що таке migration?

Міграція бази даних - це версіонований програмний сценарій, що фіксує та контролює зміни схеми бази даних (створення й зміну таблиць, колонок, індексів) у часі. Міграції дозволяють автоматизовано переводити структуру бази на новішу версію (upgrade) або повертатися до попереднього стану (downgrade).

41. Для чого використовується Alembic?

Alembic використовується як інструмент міграцій для SQLAlchemy, що відстежує зміни в ORM-моделях та підтримує схему бази даних в актуальному стані. Він забезпечує автоматичну генерацію файлів міграцій, лінійну історію ревізій та безпечне застосування змін у різних середовищах.

42. Для чого потрібна команда alembic upgrade head?

Команда alembic upgrade head послідовно застосовує всі невиконані міграції від поточної версії бази даних до найсвіжішої доступної ревізії (head). Це синхронізує структуру таблиць реальної бази даних з останнім кодом моделей проєкту.

43. Що таке integration test?

Інтеграційний тест - це тест, який перевіряє коректність спільної взаємодії кількох компонентів системи або роботу коду зі справжніми зовнішніми сервісами, такими як реляційна СУБД. Він перевіряє реальні сценарії передачі та обробки даних між шарами архітектури.

44. Чим integration test відрізняється від unit test?

Unit-тест перевіряє невелику одиницю поведінки, наприклад функцію або правило класу, з контрольованими залежностями. Моки можуть бути корисними, але не є обов'язковими, а швидкість не визначає тип тесту. Integration test перевіряє взаємодію компонентів; у цій роботі це реальні SQLite, ORM, Repository і Service разом. Такі тести зазвичай потребують додаткової підготовки та очищення тестової бази.

45. Навіщо використовувати окрему test database?

Використання окремої тестової бази гарантує ізоляцію тестів від робочих або розробницьких даних, запобігаючи їх пошкодженню чи видаленню. Це забезпечує чистий і відтворюваний початковий стан для кожного прогону тестів.

46. Що перевіряє тест rollback?

Тест rollback перевіряє, що у разі виникнення винятку або помилки посеред транзакції всі проміжні зміни коректно скасовуються, не залишаючи сліду в базі даних. Це підтверджує дотримання атомарності та збереження узгодженого стану даних.

47. Для чого використовуються fixtures?

Фікстури у pytest використовуються для підготовки та очищення тестового середовища: ініціалізації бази даних, створення з'єднань, керування транзакційними сесіями та генерації тестових сутностей. Вони запобігають дублюванню коду налаштувань і забезпечують незалежність тестів.

48. Що робить SQL JOIN?

Оператор SQL JOIN об'єднує рядки з двох або більше таблиць у єдиний результуючий набір на основі логічної умови співставлення їхніх спільних колонок (зазвичай зовнішнього та первинного ключів).

49. Що робить GROUP BY?

Речення GROUP BY об'єднує рядки з однаковими значеннями у вказаних стовпцях в окремі групи для виконання над ними групових розрахунків. Воно використовується разом з агрегатними функціями для отримання узагальненої статистики по категоріях.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX?

Ці функції використовуються для виконання агрегатних розрахунків над наборами значень у стовпцях: COUNT підраховує кількість рядків, SUM обчислює загальну суму, AVG знаходить середнє значення, а MIN і MAX визначають відповідно мінімальне та максимальне значення у вибірці.

Висновок:

Додав до системи обліку студентів SQLite, SQLAlchemy ORM, Repository та сервіс із транзакційним масовим переведенням. Перевірив CRUD, фільтрацію, агрегації, безпечний DB-API пошук та rollback. Міграція Alembic додає email без втрати наявних записів. У збереженому запуску pytest -v успішно пройшли 40 тестів persistence layer.